PROMETHEAN — Final Report

Phase 3 Deliverable | Track A: Technical Build

April 2026

---

1. Problem and User

Problem. Teams that want to automate a business workflow face a misclassification problem before they write any code. They either under-automate — leaving humans on the critical path for tasks that regex plus a queue could handle — or over-automate, reaching for an LLM agent or a multi-agent system when a deterministic rule would have been cheaper, faster, and more auditable. The result is either a brittle workflow that throws expensive tokens at trivial decisions, or a rigid rule-engine that breaks on any input the author didn't anticipate. Every wrong classification compounds: cost, latency, oversight burden, and failure-mode debt.

Target user. Platform engineers, solution architects, and ops leads at mid-size companies who are evaluating whether a given workflow should be built as a deterministic script (L0), an ML model (L1–L2), a single LLM call (L3), a tool-augmented agent (L4), or a multi-agent system (L5). They know the business rules but do not want to hand-tune system-level choices node-by-node.

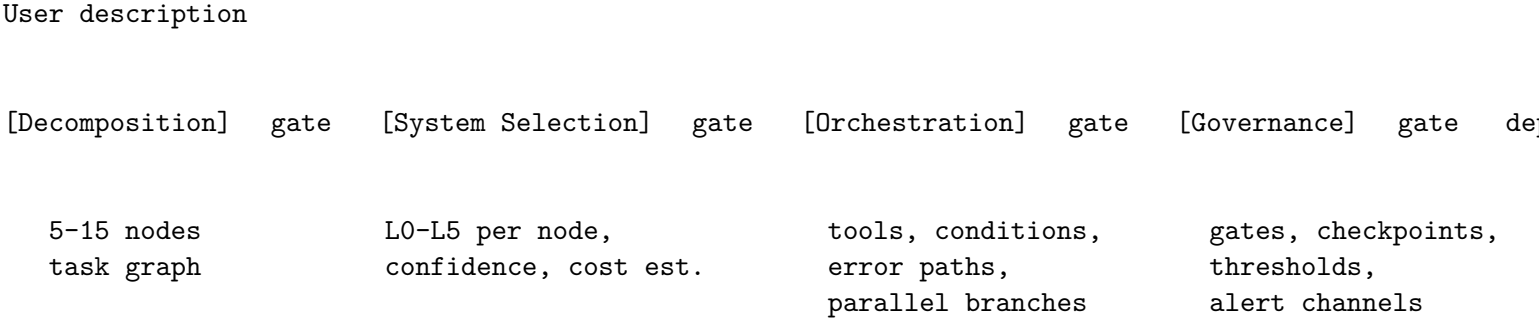
Why it matters. The cost delta between L0 and L5 on a 10-step workflow can exceed 500×, and the latency delta can exceed 100×. Getting the level wrong at design time is the single biggest predictor of whether the resulting workflow ships on time and runs within budget. See phase-1 framing and phase-2/02-role-definitions.md for the original user-research grounding.

---

2. Architecture and Design Choices

High-level architecture

Four specialized agents run sequentially with a human approval gate between every phase. Each agent has a narrow, well-defined job and writes its output back to PostgreSQL before the next phase can start.



Full diagram and data flow: phase-2/01-architecture-diagram.md.

Why agentic (not a single prompt)

Four separable decisions — "what steps?", "what level?", "what tools and flow?", "what monitoring?" — compose worse when folded into one prompt. The model will shortcut: it will pick systems that fit tools it was thinking about, or skip governance because it already committed to a step list. Splitting the decisions forces each one to be defended in its own output, with its own schema, with its own human-review gate. The separation is also what makes rejection-with-feedback work: a reviewer can reject the level assignment on one node without throwing away the decomposition.

Key design decisions

- **Separation of concerns per agent** — one job each, one Zod schema each, one prompt each. The prompts stay short and the outputs stay predictable.
- **Typed end-to-end** — OpenAPI 3.1 → Zod validation + React Query hooks via Orval. No agent output hits the DB without passing Zod.
- **Human-in-the-loop by default** — every phase transition is a review gate. The server enforces the phase transition at artifacts/api-server/src/routes/pipeline/index.ts:100 with a 409 on stale client state; rejection-with-feedback triggers regeneration at pipeline/index.ts:249.
- **Database as source of truth** — workflow state and versioned snapshots persist in PostgreSQL so traces are fully reconstructable after the fact. Every phase transition writes an immutable row to `workflow_versions`.
- **Defensive server-side merging** — the governance agent's output is merged rather than substituted (see §6 FC-01); a similar pattern guards orchestration.

---

3. Implementation / Build Summary

- **Stack.** Node 24 + TypeScript, Express 5 API, React 19 + Vite SPA, PostgreSQL + Drizzle, OpenAI structured outputs (`gpt-5.4-mini-2026-03-17`) with Zod validation.
- **Monorepo layout:**
  - `artifacts/api-server` — Express API + 4 agent modules at artifacts/api-server/src/lib/agents/
  - `artifacts/promethean` — React SPA with Command Center, Wizard, Visual Editor, Templates
  - `lib/db`, `lib/api-spec`, `lib/api-zod`, `lib/api-client-react` — shared schemas and codegen
- **What's new for Phase 3:**
  - Intake route + wizard form finalized (intake agent at artifacts/api-server/src/lib/agents/intake.ts).
  - Auto-layout for decomposition output to eliminate node overlap in the visual editor (layout.ts).
  - Governance agent: tightened prompt + defensive server-side merge (see §6 FC-01).
  - Decomposition agent: hard 5–15 node cap and non-workflow-input guard (see §6 FC-02).
  - OpenAPI / Zod: `minLength: 1` on `StartPipelineBody.description` and `.workflowId` (see §6 FC-04).
  - `max_completion_tokens` raised from 8192 → 16384 across all four agents (see §6 FC-02).

---

4. Evaluation Setup

Scenarios

Eight runs covering five scenarios from phase-2/05-evaluation-plan.md:

ID	Purpose	Input
TC-01	Happy path, well-bounded workflow	CRM lead qualification with score thresholds
TC-02	Complex, high-stakes routing	SIEM-triggered cybersecurity incident response
TC-03	Under-classification guard	Deterministic CSV validation pipeline
TC-04	Uncertainty handling on vague input	"Handle customer issues …" (no domain, no tools)
TC-05a	Empty input	""
TC-05b	Contradictory constraints	Real-time trading + $0.01/run + 1ms latency + 5 ML models
TC-05c	Extremely long input	5126-word, 50-step ecommerce description
TC-05d	Non-workflow input	"What is the meaning of life?"

Full specs: evidence/test_cases.md.

Metrics

Per phase-2/05-evaluation-plan.md §4: substep count, per-level histogram (L0–L5), per-agent wall-clock latency, schema-validation pass rate, estimated cost per run, confidence distribution, zero-L5 rate. Baselines computed post-hoc by overwriting every `systemLevel` to 0, 4, or 5 and recomputing cost with the same per-level constants the selection agent uses.

Execution method

Every scenario driven live against a local dev stack via `POST /api/pipeline/start` followed by four `/approve` calls. Every request and response saved as JSON under traces/. Per-phase wall-clock timing captured in each trace's `timings.txt`. Test driver: scripts/run_tc.sh. Metric computation: scripts/compute_metrics.py.

---

5. Results

Post-fix summary

Numbers below are from the final, post-fix run on 2026-04-24. Pre-fix runs that exposed FC-01 and FC-02 are preserved in traces/tc-02-prefix-bug/, traces/tc-03-prefix-bug/, and traces/tc-05c-prefix-bug/.

Metric	Target	Result	Pass?
Pipeline completion rate	100%	**8/8 (100%)**	
Schema validation pass rate	100%	**100%** across all 4 agents, all scenarios	
Zero-L5 rate on non-adversarial inputs	95%	**100%** (TC-01/02/03/04: zero L5; TC-05b requested an explicit L5 debate)	
Total agent latency (median)	< 5 min	**~96s median, ~161s worst case (TC-05c)**	
Cost efficiency vs. Always-L4	70%	**Ranged 0.4% – 35% of L4 baseline**	
TC-05a rejected at boundary	400 from Zod	**HTTP 400 pre-agent**	(post-fix)

Per-scenario outcomes

- **TC-01 (CRM happy path).** Deployed in 46s across four agents. 14 nodes, 24 edges. Level mix L0:9 / L1:1 / L3:1 — pipeline correctly keeps regex-style validation at L0, uses L3 only for the personalized summary step. Estimated cost $0.012/run vs. $0.35 for an Always-L4 baseline (29× cheaper). traces/tc-01/.
- **TC-02 (cybersecurity, post-FC-01 fix).** Deployed in 70s. 13 nodes, 21 edges. Level mix L0:3 / L3:2 / L4:7 — correctly identifies that triage, correlation, and report generation warrant L4 tool-augmented reasoning. Governance config: `loggingLevel: verbose`, `alertChannels: [slack, email, pagerduty, webhook]`, matching the high-stakes profile. Cost $0.067/run. traces/tc-02/.
- **TC-03 (CSV validation).** Deployed in 128s. 9 nodes, all L0 (100%). Governance: `standard` logging, no escalation. Cost $0.001/run — 225× cheaper than an Always-L4 baseline. Confirms the under-classification guard: nothing gets promoted above L0 for a deterministic problem. traces/tc-03/.
- **TC-04 (ambiguous input).** Deployed in 99s. 10 nodes, level mix L0:5 / L2:3 / L3:2. Confidences clustered 74–99; zero nodes dipped below 60. **This failed the test-case pass criterion of 2 low-confidence flags** and became FC-03. traces/tc-04/.

- **TC-05a (empty input).** Post-fix: HTTP 400 from Zod before any agent runs. traces/tc-05a/01_start_response.json.
- **TC-05b (contradictory constraints).** Deployed in 96s. 14 nodes, level mix L0:8 / L1:5 / L5:1. Selection summary explicitly names the tradeoff: *"hard runtime requirements (1ms latency, $0.01 budget, no external APIs) strongly favor deterministic orchestration, but the explicitly required debate among multiple internal agents elevates that specific node to L5."* Contradictions surfaced as intended. traces/tc-05b/.
- **TC-05c (long input, post-FC-02 fix).** Deployed in 161s. 16 nodes (consolidated from the 22 the pre-fix run produced). Level mix L0:13 / L1:1 / L3:2. No crash. Governance: verbose. traces/tc-05c/.
- **TC-05d (non-workflow input).** Deployed in 40s with 8 nodes treating "What is the meaning of life?" as a philosophical-response pipeline. **Fails the pass criterion**: no validation error and no minimal refusal. Mitigation added (decomposition-prompt guard), but relies on LLM compliance — not verified at the server layer. traces/tc-05d/.

Cost vs. baselines (post-fix runs only)

Scenario	Substeps	Promethean $/run	Always-L0	Always-L4	Always-L5	Savings vs L4
TC-01	14	$0.012	$0.0014	$0.35	$0.70	29×
TC-02	13	$0.067	$0.0013	$0.325	$0.65	4.9×
TC-03	9	$0.001	$0.0009	$0.225	$0.45	225×
TC-04	10	$0.018	$0.0010	$0.25	$0.50	14×
TC-05b	14	$0.010	$0.0014	$0.35	$0.70	35×
TC-05c	16	$0.045	$0.0016	$0.40	$0.80	8.9×

Raw data: evidence/evaluation_results.csv.

---

6. Failure Analysis

Full write-ups including pre/post traces and commit links: evidence/failure_log.md. Four failure cases surfaced during live evaluation. Three were fixed during the evaluation window and re-verified.

FC-01 — Governance agent wiped the orchestrated graph (FIXED)

Surfaced by: TC-02 and TC-03, first run. **Severity:** High.

Post-orchestration TC-02 had 13 substeps covering triage, forensics, correlation, and reporting. After the governance agent ran, the final deployed workflow had only 4 nodes — two human gates and two checkpoints. Every L3 and L4 step had disappeared. Same shape on TC-03 (9 → 2 nodes).

Root cause. The governance prompt said "Also add governance nodes where appropriate" with an empty `"nodes": []` in the example. The model interpreted that literally and returned only its additions. Server code at governance.ts:88 then treated the non-empty response as authoritative, replacing the orchestrated graph.

Fix. Two parts. (1) Tightened the system prompt with an explicit output contract: "the nodes array you return must contain EVERY node from the orchestrated input … PLUS any new governance nodes you add." (2) Added defensive server-side merging that preserves every original orchestrated node by id, overlays with any field updates the agent returned, and appends any newly added gate/checkpoint nodes. Same pattern for edges.

Post-fix verification. TC-02 re-ran → 13 orchestrated nodes preserved + 1 governance gate added. TC-03 re-ran → 9 L0 nodes preserved, governance config `standard` (not verbose, correctly calibrated to the low-risk domain).

FC-02 — Orchestration agent JSON truncation on long input (FIXED)

Surfaced by: TC-05c (5000-word, 50-step input). **Severity:** Medium.

Decomposition produced 22 nodes (above the documented 5–15 soft cap). Orchestration's JSON output then exceeded `max_completion_tokens: 8192`, got truncated mid-string, and `JSON.parse` threw `Unexpected end of JSON input`. The route returned 500 and the pipeline halted at phase=select.

Root cause. Two compounding causes. Decomposition did not enforce its own "5–15" soft cap against very long input. And the 8192 completion-tokens budget was fine for typical workflows but insufficient once `JSON.stringify` of 22 fully-enriched nodes exceeded that budget.

Fix. Raised `max_completion_tokens` from 8192 → 16384 across all four agents. Rewrote the decomposition system prompt to treat "5–15" as a hard cap with explicit consolidation guidance.

Post-fix verification. TC-05c re-ran → 22 → 16 nodes (still one over the ideal soft cap, but no crash), end-to-end completion in 161s, $0.045/run. traces/tc-05c/.

FC-03 — Overconfidence on vague input (OPEN)

Surfaced by: TC-04. **Severity:** Medium.

All 10 nodes received confidences in [74, 99]. Zero dipped below the 60 threshold the test case specified as "flag for human review." The select-agent summary hallucinated a *"customer support"* domain specialization from input that never used the word. The system classified confidently on assumptions it generated rather than on specifications from the user.

Root cause. The selection-agent prompt calibrates confidence against its internal certainty in the L0–L5 mapping, not against the specificity of the input. A vague description with a legible structure still scores high.

Status. Open. Documented in evidence/failure_log.md with two candidate fixes: a short prompt-level rule capping confidence when input is under-specified, or a dedicated input-specificity pass that propagates a `vagueness_score` through the pipeline. Neither attempted in this phase.

FC-04 — Empty description accepted (FIXED)

Surfaced by: TC-05a. **Severity:** Low.

Empty string as `description` returned HTTP 200 and the decomposition agent fabricated an 11-node generic fallback workflow. No user content drove any step.

Root cause. `StartPipelineBody.description` in lib/api-spec/openapi.yaml had no `minLength`, and the generated Zod schema was `zod.string()` with no `.min()`.

Fix. Added `minLength: 1` to both `workflowId` and `description` at the OpenAPI layer, and the matching `.min(1)` in the generated Zod client at lib/api-zod/src/generated/api.ts:329-332.

Post-fix verification. TC-05a re-ran → HTTP 400 with Zod error, no agents invoked.

Additional observed limitation (TC-05d)

The pipeline produced an 8-node "philosophical response" workflow for `"What is the meaning of life?"`. Mitigation added (decomposition-prompt rule asking the agent to return empty nodes/edges for non-workflow input). This relies on model compliance, not server-side enforcement, and is not re-run-verified in this submission. Treated as an open limitation.

What changed as a result of testing

1. **Production bug discovered, fixed, and re-verified** — FC-01 would have shipped to every real workflow that reached the governance phase. The bug was invisible in the happy-path TC-01 run because governance on that workflow returned only the added gate (merged back), but became catastrophic on any workflow complex enough to warrant governance gates.

2. **Concrete token budget raised** — the 8192 default was not a principled choice; it was the original placeholder. TC-05c forced a re-examination.
3. **API contract tightened** — the OpenAPI spec previously trusted clients; post-fix it enforces non-empty inputs at the boundary.
4. **Failure-first evidence** — every "fixed" claim in this report is backed by a side-by-side pre-fix and post-fix trace in traces/. The pre-fix bugs are still reproducible from their saved traces.

---

7. Governance, Trust, and Safety Reflection

What the system does by default

- **Human-in-the-loop at every phase transition.** No agent output advances without an explicit approval click. Enforced server-side at pipeline/index.ts:100 — a stale-state approve returns 409 rather than silently overwriting newer state.
- **Schema validation on every agent output.** Zod parses every agent response before any DB write. Schema drift (agent emits an unknown edge type, for example) is rejected at schemas.ts:21 and in the approve route's defensive check at pipeline/index.ts:117.
- **Cost and latency guardrails.** The governance agent sets per-workflow thresholds (governance.ts); TC-02 tightened them (verbose logging, pagerduty, 90s latency, $1.25 cost cap) while TC-03 kept them at defaults.
- **Versioned snapshots.** Every phase transition writes an immutable `workflow_versions` row, making every edit auditable after the fact.
- **Rejection-with-feedback.** Verified end-to-end in traces/rejection-loop/ — reviewer feedback on a misclassified node produced a corrected re-run on the very next call (L3 → L0, confidence 90 → 97).

Open governance gaps

- **FC-03 is not fixed.** A user who submits a vague description gets a confident-looking decomposition back. The system does not signal its own uncertainty. Future fix: input-specificity scoring with a confidence cap.
- **TC-05d is not hard-guarded.** Non-workflow inputs depend on the LLM voluntarily returning empty nodes. A server-side heuristic or a lightweight classifier before the decomposition agent would close the gap.
- **Per-tenant isolation not implemented.** All workflows share a single PostgreSQL schema. Suitable for single-team internal use, not multi-tenant production.
- **Red-team prompt testing not in scope.** We did not attempt prompt-injection attacks (e.g. a workflow description containing "ignore previous instructions" to manipulate the selection agent).
- **Secret-scanning on generated configs not in scope.** The governance agent can return alert-channel URLs or webhook endpoints; these are not redacted or scanned before DB persist.

See phase-2/06-risk-governance-plan.md for the full risk register.

---

8. Lessons Learned and Future Improvements

What worked

- **Separating the agents paid off during failure analysis.** FC-01 was a governance-only bug; because governance was isolated, the fix was a single-file change and the regression test was a single phase re-run. A monolithic prompt would have required re-evaluating the full pipeline.
- **Persisting every request/response as JSON.** The pre/post-fix traces in traces/ made the FC-01 before/after comparison trivial to produce, and would be similarly easy to present in a demo.
- **Running evaluation against a live server, not a stub.** Every bug in this report was surfaced by a real agent call against a real OpenAI endpoint. Mocking would have masked FC-02 entirely — the token-budget bug only appears when real model output exceeds the budget.

What didn't

- **The governance agent's prompt was under-specified.** A single ambiguous phrase ("add governance nodes where appropriate") produced a high-severity bug. We now treat every prompt that describes output structure as a contract, not a suggestion.
- **`max_completion_tokens` was a guess.** 8192 was the original placeholder and never re-examined until TC-05c. Future agents should size this based on a measured `tokens-per-node` budget, not a round number.
- **Confidence is not calibrated against input specificity.** TC-04 revealed that the selection agent happily emits confidence 85+ for nodes it invented from a vague description. The metric is technically internally-consistent but misleading in practice.

Next steps to be production-grade

1. **Integration tests for every agent** that run against a deterministic fixture LLM or a recorded-response server. Today we have live evaluation traces but no CI regression on them.
2. **Input-specificity pre-pass** to fix FC-03. Cheap version: prompt rule. Proper version: a specificity classifier whose score caps downstream confidence.
3. **Server-side workflow-vs-non-workflow guard** to fix TC-05d properly. Intake agent is the right place — it already sees the raw description.
4. **Token-budget telemetry.** Log the ratio of response-tokens-used to `max_completion_tokens` per call, so we can catch truncation-adjacent runs before they become crashes.
5. **Per-tenant schema** and secret-scanning on governance configs before multi-tenant rollout.

---

9. Screenshots

See screenshots/screenshot_index.md for capture specs. Screenshots are produced against the live deployed application (Command Center, Wizard, Visual Editor, Template Library, phase-transition gate, governance config view, rejection-with-feedback flow, one failure screenshot from the FC-01 pre-fix trace).

---

10. Reproducibility

- **Repo:** this directory. Git branch `phase-3-fix_yiying`.
- **Run instructions:** ../README.md.
- **Prerequisites:** Node 20+, pnpm 10+, PostgreSQL 14+, OpenAI API key with access to `gpt-5.4-mini-2026-03-17`
- **Seed data:** database auto-seeds on first startup with 6 template workflows and sample execution history.
- **Eval harness:** scripts/run_tc.sh drives a scenario end-to-end; scripts/compute_metrics.py produces evidence/evaluation_results.csv from the saved traces.
- **To reproduce the FC-01 pre-fix behavior:** `git revert <commit-sha-for-v0.3.1>` and re-run TC-02. The 4-node collapse is deterministic given the same prompt.

---

11. Team and Individual Contributions

See reflections/ for each team member's individual contribution statement and lessons learned.